15,471,951 members Sign in

articles Q&A forums stuff lounge ? Search for articles, questions,

A Simple UDP Messaging Client Toolkit



Andre Trollip

15 Oct 2010 CPOL

Rate me: 4.60/5 (4 votes)

A simple set of classes to make creating Peer to Peer messaging easy using UDP. Features built-in message concatenation and delivery receipts for reliable transmission.

Download source code - 12.79 KB

Introduction

Most of us at some stage in our careers have had to, or was interested in developing some application that involved TCP or UDP networking. I have had to do a few in my 10 years in software development. When I was required to do another such utility, I realised soon after starting the design, that I could say deja vu to a lot of the features required. I decided to build something that will hopefully be useful (if not sufficient) in most of my future network utility applications.

From the outset, I had four design objectives:

Ease of Use

I wanted to make this toolkit useable for all levels of developers. I am exposed to a wide range of developers when it comes to experience and skill level. Making the toolkit easy to use gave me a greater degree of reuse as a rather junior developer should be able to use this library just as easily as a very senior developer.

I achieved the ease of use by keeping the number of publicly exposed methods to a minimum. I also added numerous overloads to the basic operations, e.g. sending a message and instantiating the client.

Robustness

Having reuse in mind, the toolkit needed to be robust. For instance, using it in a peer-to-peer environment as well as in a client-server environment should require no special knowledge of either methodology. Any data can be sent and there are no explicit limitations on size. Sending finite messages or streaming can both be achieved.

The user can implement his own network protocol that *piggybacks* on the underlying *protocol*.

Message Concatenation

A very important feature for me was for the toolkit to split up large messages into chunks of a specified size, sending them and then be concatenated again before being presented to the consumer. This feature is hidden to the consumer and will automatically happen, depending on the packet size that the user chooses.

Packet Reliability

UDP is inherently an unreliable protocol in that whether or not a network packet is received, cannot be guaranteed. Being easy to use meant that message reliability must be optional as well as hidden from the consumer. I implemented a delivery receipt protocol for messages that are specified as needing reliability. This takes away the necessity for the developer to implement his/her own delivery confirmation mechanism.

Points of Interest

Included is a `BitConverter` class. This class uses the standard .NET `BitConverter`, but checks the endianness of the environment. It may happen that network packets are sent between hosts that do not both use Little Endian bit formats for example. The `BitConverter` class always converts the operands to **big endian**.

For sending operations, the built-in .NET `ThreadPool` is used. Thread pooling provides an easy to use, optimised method for queuing work. It was left up to the `ThreadPool` object to decide on the correct number of threads to use.

The receiving of packets is achieved with a separate background thread dedicated to listening for incoming packets.

To further achieve the ease of use, I used two events to notify the consumer of a message that was sent or received. More about this later.

Using the Code

The classes included in the project are very simple and very self-explanatory. The only class that needs to be instantiated is the `Client` class. To send your first message, simply instantiate the `Client` class and call the appropriate `EnqueueMessage(...)` method.

Here is some code to show how to use the toolkit.

C#

```
Client client;

private void MainForm_Load(object sender, EventArgs e)
{
    client = new Client(new IPEndPoint(IPAddress.Parse("127.0.0.1"), 5), 1024, 10);

    client.MessageSent += new OnMessageSent(client_MessageSent);
    client.MessageReceived += new OnMessageReceived(client_MessageReceived);
}
```

In the snippet above, the Windows Form's `Load` event is used to instantiate the `Client` class. For this particular overload, the *localhost* I.P. address is passed as the local end point. The second parameter is the packet size. This number is the number of bytes into which messages will be broken up into if the sent message is larger than the number. For instance, if the message you send is 3092 bytes long, the message will be sent as three packets of 1024 bytes and a fourth packet of 20 bytes. The third parameter is the time in seconds that the client will wait after sending a packet, before assuming the packet has failed and resending it. The default number of times a packet will be attempted to send is three. This value can be overridden using a different overload for the constructor.

The two events of the `Client` class are bound to methods here. `OnMessageSent` is bound to `client_MessageSent();` and `OnMessageReceived` is bound to `OnMessageReceived();`. These events are raised after a message has been sent and received respectively.

C#

```
void client_MessageReceived(object sender, MessageReceivedEventArgs args)
{
    this.Invoke(new MethodInvoker(delegate
    {
    ReceivedMessageTextBox.Text = "\r\n\r\n" +
        System.Text.Encoding.ASCII.GetString(args.Data);
    }));
}
```

In the implementation of the event method above, I simply add the received message's text to a text box on the Windows Form.

Of course, this wouldn't make sense if for instance a binary file was sent as the message. The `MessageReceivedEventArgs` passed to the method contains the data that was received as a byte array.

C# Shrink ▲

```
void client_MessageSent(object sender, MessageSentEventArgs args)
{
    switch (args.Status)
    {
    case SendStatus.Failed:
        this.Invoke(new MethodInvoker(delegate
        {
        ActivityListBox.Items.Add("Failed message: " +
                    args.MessageID.ToString());
        }));
        break;

    case SendStatus.Sent:
        this.Invoke(new MethodInvoker(delegate
        {
        ActivityListBox.Items.Add("Sent message: " + args.MessageID.ToString());
        }));
        break;

    case SendStatus.Delivered:
        this.Invoke(new MethodInvoker(delegate
        {
        ActivityListBox.Items.Add("Delivered message: " +
                    args.MessageID.ToString());
        }));
        break;
    }
}
```

When a message was sent not specifying the reliable flag, this event will be raised immediately after sending the message. The `MessageSentEventArgs` contains a field called `SendStatus`. This field will be set to `Sent` indicating the message was sent, but no further assumption can be made as to reaching its destination.

If the message was sent reliably, that is, sending with the `Reliable` flag set to `true`, the `MessageSentEventArgs` will have either the value `Delivered` or `Failed` in the `SendStatus` field. When the value is `Delivered`, the consumer can then safely assume the entire message was received by the recipient. Conversely, when the value is `Failed`, the message was sent, but no delivery confirmation was received. The consumer should in this case assume the message failed.

C#

```
private void SendButton_Click(object sender, EventArgs e)
{
    ActivityListBox.Items.Add("Attempt Send message: " +
        client.QueueMessage(System.Text.Encoding.ASCII.GetBytes(ToSendTextBox.Text),
        64, new System.Net.IPEndPoint(IPAddress.Parse(textBox2.Text), 5), 5,
        checkBox1.Checked).ToString());
}
```

The event above is the implementation of a button click event. In here, I call the `client.QueueMessage();` method. This method is so called as messages are sent asynchronously and

thus queued until they are ready to be sent.

History

- 2010-10-04 -- Created article

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

Andre Trollip

Software Developer (Senior)

South Africa

B.Sc (Comp. Science)
17 years software development experience.

Adventure junkie & keen golfer.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

My vote of 5

ndoro75 2-Nov-10 4:45

Re: My vote of 5

Andre Trollip 3-Nov-10 12:05

A bug and a resolution? Short messages "disappear"!

Mark Long 15-Oct-10 5:07

Re: A bug and a resolution? Short messages "disappear"!

Andre Trollip 15-Oct-10 5:44

Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2010 by Andre Trollip
Everything else Copyright © CodeProject, 1999-2022

Web04 2.8:2022-10-01:1